CMPSC/DS 442: Homework 3

TO PREPARE AND SUBMIT HOMEWORK

Follow these steps exactly, so the Gradescope autograder can grade your homework. Failure to do so will result in a zero grade:

1. You **must** download the homework template file `homework3_starter.py` from Canvas. Each template file is a python file that gives you a headstart in creating your homework python script with the correct function names for autograding. For this homework, you will also need to download these files from Canvas:
   `homework3_dominoes_game_gui.py`
   `sudoku.zip`

2. You **must** rename the file by replacing `starter` with your PSU id from your official PSU. For example, if your PSU email id is abcd1234, you would rename your file as `homework3_abcd1234.py`.

4. Upload your `homework3_abcd1234.py` file to Gradescope by the due date.

5. Make sure your file can be imported before you submit; the autograder imports your file. If it won't import, you will get a zero.

6. Please ask questions if you have any via piazza or during office hours.

Instructions

In this assignment, you will implement the Domino game and algorithms for the popular puzzle game Sudoku.

A skeleton *.py file `homework3_starter.py` containing empty definitions for each question has been provided (see above). Do not modify any names or function signatures in the provided template file, or the autograder cannot test your functions. However, you are free to introduce additional variables or functions if needed.

You may import definitions from any standard Python library, and are encouraged to do so in case you find yourself reinventing the wheel. If you are unsure where to start, consider taking a look at the data structures and functions defined in the `collections`, `itertools`, `copy`, `Queue`, and `random` modules. You cannot use any libraries (like numpy) that are not standard; see the homework FAQ on Canvas

You will find that in addition to a problem specification, most programming questions also include a pair of examples from the Python interpreter. These are meant to illustrate typical use cases, and must not be taken as comprehensive test suites.

1. Dominoes Game [30 points]

In this section, you will develop an AI for a game in which two players take turns placing *1 x 2* dominoes on a rectangular grid. One player must always place his dominoes vertically, and the other must always place his dominoes horizontally. The last player who successfully places a domino on the board wins.

As with the Tile Puzzle, an infrastructure that is compatible with the provided GUI has been suggested. However, only the search method will be tested, so you are free to choose a different approach if you find it more convenient to do so.

The representation used for this puzzle is a two-dimensional list of Boolean values, where `True` corresponds to a filled square and `False` corresponds to an empty square.

1. **[0 points]** In the `DominoesGame` class, write an initialization method `_init_(self, board)` that stores an input board of the form described above for future use. You additionally may wish to store the dimensions of the board as separate internal variables, though this is not required.

2. **[0 points]** *Suggested infrastructure.*

   In the `DominoesGame` class, write a method `get_board(self)` that returns the internal representation of the board stored during initialization.

```
>>> b = [[False, False], [False, False]]
>>> g = DominoesGame(b)
>>> g.get_board()
[[False, False], [False, False]]
```

```
>>> b = [[True, False], [True, False]]
>>> g = DominoesGame(b)
>>> g.get_board()
[[True, False], [True, False]]
```

   Write a top-level function `make_dominoes_game(rows, cols)` that returns a new `DominoesGame` of the specified dimensions with all squares initialized to the empty state.

```
>>> g = make_dominoes_game(2, 2)
>>> g.get_board()
[[False, False], [False, False]]
```

```
>>> g = make_dominoes_game(2, 3)
>>> g.get_board()
[[False, False, False],
 [False, False, False]]
```

   In the `DominoesGame` class, write a method `reset(self)` which resets all of the internal board's squares to the empty state.

```
>>> b = [[False, False], [False, False]]
>>> g = DominoesGame(b)
>>> g.get_board()
[[False, False], [False, False]]
>>> g.reset()
>>> g.get_board()
[[False, False], [False, False]]
```

```
>>> b = [[True, False], [True, False]]
>>> g = DominoesGame(b)
>>> g.get_board()
[[True, False], [True, False]]
>>> g.reset()
>>> g.get_board()
[[False, False], [False, False]]
```

In the `DominoesGame` class, write a method `is_legal_move(self, row, col, vertical)` that returns a Boolean value indicating whether the given move can be played on the current board. A legal move must place a domino fully within bounds, and may not cover squares which have already been filled.

If the `vertical` parameter is `True`, then the current player intends to place a domino on squares `(row, col)` and `(row + 1, col)`. If the `vertical` parameter is `False`, then the current player intends to place a domino on squares `(row, col)` and `(row, col + 1)`. This convention will be followed throughout the rest of the section.

```
>>> b = [[False, False], [False, False]]
>>> g = DominoesGame(b)
>>> g.is_legal_move(0, 0, True)
True
>>> g.is_legal_move(0, 0, False)
True
```

```
>>> b = [[True, False], [True, False]]
>>> g = DominoesGame(b)
>>> g.is_legal_move(0, 0, False)
False
>>> g.is_legal_move(0, 1, True)
True
>>> g.is_legal_move(1, 1, True)
False
```

In the `DominoesGame` class, write a method `legal_moves(self, vertical)` which yields the legal moves available to the current player as (row, column) tuples. The moves must be generated in row-major order (i.e. iterating through the rows from top to bottom, and within rows from left to right), starting from the top-left corner of the board.

```
>>> g = make_dominoes_game(3, 3)
>>> list(g.legal_moves(True))
[(0, 0), (0, 1), (0, 2), (1, 0), (1, 1),
 (1, 2)]
>>> list(g.legal_moves(False))
[(0, 0), (0, 1), (1, 0), (1, 1), (2, 0),
 (2, 1)]
```

```
>>> b = [[True, False], [True, False]]
>>> g = DominoesGame(b)
>>> list(g.legal_moves(True))
[(0, 1)]
>>> list(g.legal_moves(False))
[]
```

In the `DominoesGame` class, write a method `execute_move(self, row, col, vertical)` which fills the squares covered by a domino placed at the given location in the specified orientation.

```
>>> g = make_dominoes_game(3, 3)
>>> g.execute_move(0, 1, True)
>>> g.get_board()
[[False, True, False],
 [False, True, False],
 [False, False, False]]
```

```
>>> g = make_dominoes_game(3, 3)
>>> g.execute_move(1, 0, False)
>>> g.get_board()
[[False, False, False],
 [True, True, False],
 [False, False, False]]
```

In the `DominoesGame` class, write a method `game_over(self, vertical)` that returns whether the current player is unable to place any dominoes.

```
>>> b = [[False, False], [False, False]]
>>> g = DominoesGame(b)
>>> g.game_over(True)
False
>>> g.game_over(False)
False
```

```
>>> b = [[True, False], [True, False]]
>>> g = DominoesGame(b)
>>> g.game_over(True)
False
>>> g.game_over(False)
True
```

In the `DominoesGame` class, write a method `copy(self)` that returns a new `DominoesGame` object initialized with a deep copy of the current board. Changes made to the original puzzle must not be reflected in the copy, and vice versa.

```
>>> g = make_dominoes_game(4, 4)
>>> g2 = g.copy()
>>> g.get_board() == g2.get_board()
True
```

```
>>> g = make_dominoes_game(4, 4)
>>> g2 = g.copy()
>>> g.execute_move(0, 0, True)
>>> g.get_board() == g2.get_board()
False
```

In the `DominoesGame` class, write a method `successors(self, vertical)` that yields all successors of the puzzle for the current player as (move, new-game) tuples, where moves themselves are (row, column) tuples. The second element of each successor must be a new `DominoesGame` object whose board is the result of applying the corresponding move to the current board. The successors must be generated in the same order in which moves are produced by the `legal_moves(self, vertical)` method.

```
>>> b = [[False, False], [False, False]]
>>> g = DominoesGame(b)
>>> for m, new_g in g.successors(True):
...  print m, new_g.get_board()
...
(0, 0) [[True, False], [True, False]]
(0, 1) [[False, True], [False, True]]
```

```
>>> b = [[True, False], [True, False]]
>>> g = DominoesGame(b)
>>> for m, new_g in g.successors(True):
...  print m, new_g.get_board()
...
(0, 1) [[True, True], [True, True]]
```

Optional.

In the `DominoesGame` class, write a method `get_random_move(self, vertical)` which returns a random legal move for the current player as a (row, column) tuple. *Hint: The `random` module contains a function `random.choice(seq)` which returns a random element from its input sequence.*

3. **[30 points]** In the `DominoesGame` class, write a method `get_best_move(self, limit, vertical)` which returns a 3-element tuple containing the best move for the current player as a (row, column) tuple, its associated value, and the number of leaf nodes visited during the search. Recall that if the `vertical` parameter is `True`, then the current player intends to place a domino on squares `(row, col)` and `(row + 1, col)`, and if the `vertical` parameter is `False`, then the current player intends to place a domino on squares `(row, col)` and `(row, col + 1)`. Moves must be explored in a row- major order, described in further detail above, to ensure consistency.

Your search must be a faithful implementation of the alpha-beta search given on page 170 of the course textbook, with the restriction that you must look no further than `limit` moves into the future. To evaluate a board, you must compute the number of moves available to the current player, then subtract the number of moves available to the opponent.

```
>>> b = [[False] * 3 for i in range(3)]
>>> g = DominoesGame(b)
>>> g.get_best_move(1, True)
((0, 1), 2, 6)
>>> g.get_best_move(2, True)
((0, 1), 3, 10)
```

```
>>> b = [[False] * 3 for i in range(3)]
>>> g = DominoesGame(b)
>>> g.execute_move(0, 1, True)
>>> g.get_best_move(1, False)
((2, 0), -3, 2)
>>> g.get_best_move(2, False)
((2, 0), -2, 5)
```

If you implemented the suggested infrastructure described in this section, you can play with an interactive version of the dominoes board game using the provided GUI by running the following command:

```
python homework2_dominoes_game_gui.py rows cols
```

The arguments `rows` and `cols` are positive integers designating the size of the board.

In the GUI, you can click on a square to make a move, press 'r' to perform a random move, or press a number between *1* and *9* to perform the best move found according to an alpha-beta search with that limit. The GUI is merely a wrapper around your implementations of the relevant functions, and may therefore serve as a useful visual tool for debugging.

2. Sudoku [70 points]

In the game of Sudoku, you are given a partially-filled 9 x 9 grid, grouped into a 3 x 3 grid of 3 x 3 blocks. The objective is to fill each square with a digit from 1 to 9, subject to the requirement that each row, column, and block must contain each digit exactly once.

In this section, you will implement the AC-3 constraint satisfaction algorithm for Sudoku, along with two extensions that will combine to form a complete and efficient solver.

A number of puzzles have been made available in the sudoku.zip file,

including: • An easy-difficulty puzzle: `hw3-easy.txt` .

• Four medium-difficulty puzzles: `hw3-medium1.txt` , `hw3-medium2.txt` , `hw3-medium3.txt` , and `hw3-medium4.txt` .

• Two hard-difficulty puzzles: `hw3-hard1.txt` and `hw3-hard2.txt` .

The examples in this section assume that these puzzle files have been placed in a folder named `sudoku` located in the same directory as the homework file.

An example puzzle from the Daily Pennsylvanian, available as `hw3-medium1.txt` , is depicted below.

```
* 1 5 * 2 * * * 9
* 4 * * * * 7 * *
* 2 7 * * 8 * * *
9 5 * * * 3 2 * *
7 * * * * * * * 6
* * 6 2 * * * 1 5
* * * 6 * * 9 2 *
* * 4 * * * * 8 *
  * * * 3 * 6 5 *
```

Textual Representation

	1	5		2				9
	4					7		
	2	7			8			
9	5				3	2		
7								6
		6	2				1	5
			6			9	2	
		4					8	
2				3		6	5	

Initial Configuration

6	1	5	3	2	7	8	4	9
8	4	9	5	1	6	7	3	2
3	2	7	9	4	8	5	6	1
9	5	1	4	6	3	2	7	8
7	3	2	8	5	1	4	9	6
4	8	6	2	7	9	3	1	5
1	7	3	6	8	5	9	2	4
5	6	4	7	9	2	1	8	3
2	9	8	1	3	4	6	5	7

Solved Configuration

1. **[3 points]** In this section, we will view a Sudoku puzzle not from the perspective of its grid layout, but more abstractly as a collection of cells. Accordingly, we will represent it internally as a dictionary mapping from cells, i.e. (row, column) pairs, to sets of possible values.

   In the `Sudoku` class, write an initialization method `__init__(self, board)` that stores such a mapping for future use. Also write a method `get_values(self, cell)` that returns the set of values currently available at a particular cell.

In addition, write a function `read_board(path)` that reads the board specified by the file at the given path and returns it as a dictionary. Sudoku puzzles will be represented textually as 9 lines of 9 characters each, corresponding to the rows of the board, where a digit between `"1"` and `"9"` denotes a cell containing a fixed value, and an asterisk `"*"` denotes a blank cell that could contain any digit.

```
>>> b = read_board("sudoku/hw3- medium1.txt"

>>> Sudoku(b).get_values((0, 0))
set([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
>>> b = read_board("sudoku/hw3- medium1.txt"

>>> Sudoku(b).get_values((0, 1))
set([1])
```

2. **[2 points]** Write a function `sudoku_cells()` that returns the list of all cells in a Sudoku puzzle as (row, column) pairs. The line `CELLS = sudoku_cells()` in the `Sudoku` class then creates a class-level constant `Sudoku.CELLS` that can be used wherever the full list of cells is needed. Although the function `sudoku_cells()` could still be called each time in its place, that approach results in a large amount of repeated computation and is therefore highly inefficient. The ordering of the cells within the list is not important, as long as they are all present.

```
>>> sudoku_cells()
[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), ..., (8, 5), (8, 6), (8, 7), (8, 8)]
```

3. **[5 points]** Write a function `sudoku_arcs()` that returns the list of all arcs between cells in a Sudoku puzzle corresponding to inequality constraints. In other words, each arc must be a pair of cells whose values cannot be equal in a solved puzzle. The arcs must be represented a two-tuples of cells, where cells themselves are (row, column) pairs. The line `ARCS = sudoku_arcs()` in the `Sudoku` class then creates a class-level constant `Sudoku.ARCS` that can be used wherever the full list of arcs is needed. The ordering of the arcs within the list is not important, as long as they are all present.

```
>>> ((0, 0), (0, 8)) in sudoku_arcs()
True
>>> ((0, 0), (8, 0)) in sudoku_arcs()
True
>>> ((0, 8), (0, 0)) in sudoku_arcs()
True
```

```
>>> ((0, 0), (2, 1)) in sudoku_arcs()
True
>>> ((2, 2), (0, 0)) in sudoku_arcs()
True
>>> ((2, 3), (0, 0)) in sudoku_arcs()
False
```

4. **[10 points]** In the `Sudoku` class, write a method `remove_inconsistent_values(self, cell1, cell2)` that removes any value in the set of possibilities for `cell1` for which there are no values in the set of possibilities for `cell2` satisfying the corresponding inequality constraint. Each cell argument will be a (row, column)

pair. If any values were removed, return `True`; otherwise, return `False`.

Hint: Think carefully about what this exercise is asking you to implement. How many values can be removed during a single invocation of the function?

```
>>> sudoku = Sudoku(read_board("sudoku/hw3-easy.txt")) # See below for a picture.
>>> sudoku.get_values((0, 3))
set([1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> for col in [0, 1, 4]:
...  removed = sudoku.remove_inconsistent_values((0, 3), (0, col))
...  print removed, sudoku.get_values((0, 3))
...
True set([1, 2, 3, 4, 5, 6, 7, 9])
True set([1, 3, 4, 5, 6, 7, 9])
False set([1, 3, 4, 5, 6, 7, 9])
```

5. **[10 points]** In the `Sudoku` class, write a method `infer_ac3(self)` that runs the AC-3 algorithm on the current board to narrow down each cell's set of values as much as possible. Although this will not be powerful enough to solve all Sudoku problems, it will produce a solution for easy-difficulty puzzles such as the one shown below. By "solution", we mean that there will be exactly one element in each cell's set of possible values, and that no inequality constraints will be violated.

8	2	1						7
			8				6	
	6		9	3				5
		8	2		1	6		
			7			2	8	4
2	4		6		3	7		
6		5				1		3
	7			5				
9	1	2						6

Initial Configuration

hw3-easy.txt

8	2	1	5	6	4	3	9	7
5	9	3	8	1	7	4	6	2
4	6	7	9	3	2	8	1	5
7	5	8	2	4	1	6	3	9
1	3	6	7	9	5	2	8	4
2	4	9	6	8	3	7	5	1
6	8	5	4	2	9	1	7	3
3	7	4	1	5	6	9	2	8
9	1	2	3	7	8	5	4	6

Result of Running AC-3

6. **[20 points]** Consider the outcome of running AC-3 on the medium-difficulty puzzle shown below. Although it is able to determine the values of some cells, it is unable to make significant headway on the rest.

				8	3	4		

				8	3	4	7	

3					4	8	2	1
7								
		9	4		1		8	3
4	6		5		7	1		
								7
1	2	5	3					9
		7	2	4				

Initial Configuration

hw3-medium2.txt

3					4	8	2	1
7								
		9	4		1		8	3
4	6		5		7	1		
								7
1	2	5	3					9
		7	2	4				

Inference Beyond AC-3

However, if we consider the possible placements of the digit 7 in the upper-right block, we observe that the 7 in the third row and the 7 in the final column rule out all but one square, meaning we can safely place a 7 in the indicated cell despite AC-3 being unable to make such an inference.

In the `Sudoku` class, write a method `infer_improved(self)` that runs this improved version of AC-3, using `infer_ac3(self)` as a subroutine (perhaps multiple times). You must consider what deductions can be made about a specific cell by examining the possible values for other cells in the same row, column, or block. Using this technique, you must be able to solve all of the medium-difficulty puzzles.

7. **[20 points]** Although the previous inference algorithm is an improvement over the ordinary AC-3 algorithm, it is still not powerful enough to solve all Sudoku puzzles. In the `Sudoku` class, write a method `infer_with_guessing(self)` that calls `infer_improved(self)` as a subroutine, picks an arbitrary value for a cell with multiple possibilities if one remains, and repeats. You must implement a backtracking search which reverts erroneous decisions if they result in unsolvable puzzles. For efficiency, the improved inference algorithm must be called once after each guess is made. This method must be able to solve all of the hard- difficulty puzzles, such as the one shown below.

	9		7			8	6	
	3	1			5		2	
8		6						
		7		5				6
			3		7			
5				1		7		
						1		9
	2		6			3	5	
	5	4			8		7	

Initial Configuration

hw3-hard1.txt

2	9	5	7	4	3	8	6	1
4	3	1	8	6	5	9	2	7
8	7	6	1	9	2	5	4	3
3	8	7	4	5	9	2	1	6
6	1	2	3	8	7	4	9	5
5	4	9	2	1	6	7	3	8
7	6	3	5	2	4	1	8	9
9	2	8	6	7	1	3	5	4
1	5	4	9	3	8	6	7	2

Result of Inference with Guessing